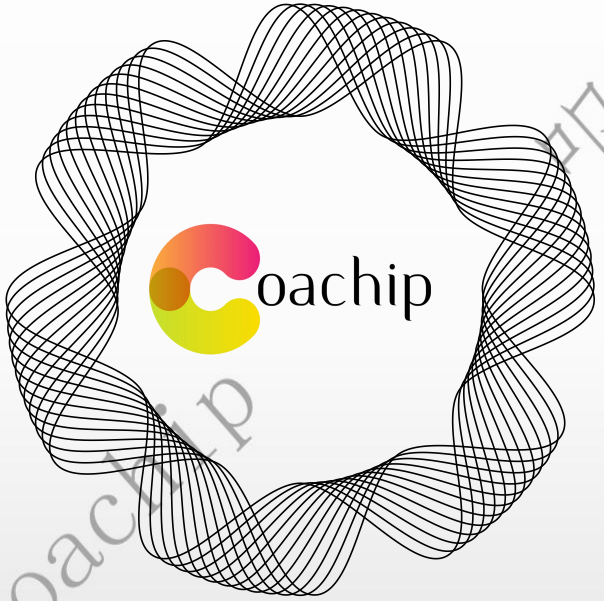



# SV面向对象编程

叩特电子Coachip

# 课程内容及目标

## 课程内容：

- 类的封装
- 类的继承

## 目标：

- 了解类的类的封装、继承




叩特电子Coachip

# 类的封装

- 封装是面向对象编程的核心思想，是指将对象的属性和行为封装起来，封装的载体是类，类通常对用户隐藏其实现的细节，这就是封装的思想。
- 采用封装的思想保证了类内部数据结构的完整性，应用该类的用户不能轻易直接操纵该数据结构，而只能执行该类允许公开的数据。这样可以避免外部对内部数据的影响，提高程序的可维护性。

叩特电子Coachip

# 类的封装—类与结构体的异同

- 二者都是容器，都可以定义数据成员；
- 类变量在声明之后，需要构造（construction）才会构建对象（object）实体，而struct在变量声明时已经开辟内存。
- 类除了可以声明数据变量成员，还可以声明方法（function/task），而struct则不能。
- 从根本来讲，结构体是用户根据实际需要定义的复合数据类型，而class是对某一类事物的抽象模板，类中不仅有数据类型属性成员，还包括对这些属性成员操作的一系列方法。

叩特电子Coachip

# 类的封装—类与模块（module）的异同

- 从数据和方法定义而言，二者均可以作为封闭的容器来定义和存储。
- 从例化来看，模块必须在仿真一开始就确定是否被例化，这可以通过 generate 来实现设计结构的变化；而对于类而言，它的变量在仿真的任何时段都可以被构造（开辟内存）创建新的对象。
- 按照硬件世界和软件世界区分的观点来看，硬件部分必须在仿真一开始就确定下来，即module和其内部过程块、变量都应该是静态的（static）；而软件部分，即类的部分可以在仿真任何阶段声明并动态创建出新的对象，这正是软件操作更为灵活的地方。
- 从封装性（encapsulation）来看，模块内的变量和方法是对外部公共（public）开放的，而类则可以根据需要来确定外部访问的权限是否是默认的公共类型、或者受保护类型（protected）还是私有类型（local）。
- 从继承性（inheritance）来看，模块没有继承性，即无法在原有module的基础上进行新module功能的扩展，如果需要之前设计的module功能则必须通过拷贝旧的module然后进行修改从而生成一个新的module的方式。而继承性是类的一大特点，不用拷贝也能使用父类所定义的方法和属性。

叩特电子Coachip

# System Verilog类的继承

```
class cat;
  protected color_t color;
  local bit is_good;
  function set_good(bit s);
    this.is_good = s;
  endfunction
endclass

class black_cat extends cat;
  function new();
    this.color = BLACK;
  endfunction
endclass
```

```
class white_cat extends cat;
  function new();
    this.color = WHITE;
  endfunction
endclass

black_cat bk;
white_cat wt;
initial begin
  bk = new();
  wt = new();
  bk.set_good(1);
  wt.set_good(1);
end
```

叩特电子Coachip

# System Verilog类的继承

- 类的继承包括了继承父类的成员变量和成员方法。也就说一个子类在继承父类成员变量的同时也继承了父类操作这些成员变量的成员方法。
- 子类在定义new函数时，需要先调用父类的new函数即**super.new()**。如果父类的new函数没有参数，子类也可以省略该调用，而系统会在编译时自动添加super.new()。
- 从对象创建时初始化的顺序来看，用户应该注意有如下的规则：
  - 子类的实例对象在初始化时首先会**调用父类的构造函数**。
  - 当父类构造函数完成时，会将子类实例对象中各个成员变量按照它们定义时显式的**默认值初始化**，如果没有默认值则不被初始化。
  - 在成员变量默认值初始化后（声明的同时即赋值），才会最后进入**用户定义的new函数中执行剩余的初始化代码**。

即特电子Coachip

# 练习

```
typedef enum {WHITE,BLACK} color_t;
class cat;
  protected color_t color;
  local bit is_good;
  function set_good(bit s);
    this.is_good = s;
  endfunction
endclass

class black_cat extends cat;
  function new();
    this.color = BLACK;
  endfunction
endclass

class white_cat extends cat;
  function new();
    this.color = WHITE;
  endfunction
endclass
```

```
black_cat bk;
white_cat wt;

initial begin
  bk = new();
  wt = new();
  bk.set_good(1);
  wt.set_good(1);
end

module test;
enum {WHITE,BLACK} color_t;
...
endmodule

vcs -sverilog -full64 test.sv
./simv
```

叩特电子Coachip

# Q&A

昂特电子Coachip